

Merge-Sort Execution

```

Merge_sort(A,le,ri) // n = ri-le+1
  if (le>=ri) return
  else
    m = floor((le+ri)/2)
    Merge_sort(A,le,m);
    Merge_sort(A,m+1,ri);
    Merge(A,le,m,ri);
    
```

Each row shows the array after each call to the Merge function finished.

— Red items were moved by Merge.

	0	1	2	3	4	5	6	7
Original	7	1	3	9	4	1	2	6
After 1 st call to Merge	1	7	3	9	4	1	2	6
After 2 nd call to Merge	1	7	3	9	4	1	2	6
After 3 rd call to Merge	1	3	7	9	4	1	2	6
	1	3	7	9	1	4	2	6
	1	3	7	9	1	4	2	6
	1	3	7	9	1	2	4	6
After last call to Merge	1	1	2	3	4	6	7	9

```

MS(0,7) // m = 3
  | MS(0,3) // m = 1
  | | MS(0,1) // m = 0
  | | | MS(0,0) //le==r, basecase
  | | | MS(1,1) //le==r
  | | | Merge(0,0,1)
  | | MS(2,3) // m = 2
  | | | MS(2,2)
  | | | MS(3,3)
  | | | Merge(2,2,3)
  | | | Merge(0,1,3)
  | MS(4,7) // m = 5
  | | MS(4,5) // m = 4
  | | | MS(4,4)
  | | | MS(5,5)
  | | | Merge(4,4,5)
  | | MS(6,7) // m = 6
  | | | MS(6,6)
  | | | MS(7,7)
  | | | Merge(6,6,7)
  | | | Merge(4,5,7)
  | | Merge(0,3,7)
    
```

Notation: $MS(le,ri)$ for $Merge_Sort(A,le,ri)$

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  void merge(int arr[], int l, int m, int r){
4
5      int i, j, k;
6      int n1 = m - l + 1;
7      int n2 = r - m;
8      int L[n1], R[n2];
9      for (i = 0; i < n1; i++)
10         L[i] = arr[l + i];
11     for (j = 0; j < n2; j++)
12         R[j] = arr[m + 1 + j];
13     i = 0;
14     j = 0;
15     k = l;
16     while (i < n1 && j < n2) {
17         if (L[i] <= R[j]) {
18             arr[k] = L[i];
19             i++;
20         }
21         else {
22             arr[k] = R[j];
23             j++;
24         }
25         k++;
26     }
27     while (i < n1) {
28         arr[k] = L[i];
29         i++;
30         k++;
31     }
32     while (j < n2) {
33         arr[k] = R[j];
34         j++;
35         k++;
36     }
37 }
38 void mergeSort(int arr[], int l, int r){
39
40     if (l < r) {
41         int m = l + (r - l) / 2;

```

```
41
42     // Sort first and second halves
43     mergeSort(arr, l, m);
44     mergeSort(arr, m + 1, r);
45
46     merge(arr, l, m, r);
47 }
48 }
49
50 int main(){
51
52     int arr[] = {38, 27, 43, 10};
53     int arr_size = sizeof(arr) / sizeof(arr[0]);
54
55     mergeSort(arr, 0, arr_size - 1);
56     int i;
57     for (i = 0; i < arr_size; i++)
58         printf("%d ", arr[i]);
59     printf("\n");
60
61     return 0;
62 }
```